

UniApp X 调研

介绍:

- uni-app x 没有使用js和webview, 它基于 uts 语言。在App端, uts在iOS编译为swift、在Android编译为kotlin。
- 开发者在 uni-app x 中, 不能编写js, 因为 uni-app x 中不自带js引擎。需使用uts, 实现跨端的同时保证最佳性能。
- uts 全称 uni type script, 是一门跨平台的、高性能的、强类型的现代编程语言。它在不同平台, 会被编译为不同平台的native语言
- uvue是一套基于uts的、兼容vue语法的、跨iOS和Android的、原生渲染引擎。

Uniapp x 集成环信SDK:

Tips:

- Uniapp x 推荐使用uts插件封装原生能力, 包括os能力或三方sdk (一般情况下, 原生库的能力是大于js库的)
- 如果你一定要使用某个js库, 还有一个办法是在uni-app x里的web-view组件, 让其运行js并返回值给uts代码。

集成 uniapp js sdk

尝试使用webview引入uniappjs sdk, 通过postMessage像应用发送sdk实例 ———— 失败, postMessage无法传递函数等

创建em-chat uts 插件 <https://github.com/Wster11/uts-em-chat/tree/main/em-chat>

集成 原生安卓SDK

- app-android config.json 通过 gradle 引入安卓SDK

```
{
  "dependencies": [
    "io.hyphenate:hyphenate-chat:3.9.4"
  ],
  "minSdkVersion": "19"
}
```

- AndroidManifest.xml 配置权限

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
package="io.dcloud.nativeresouce">
    <!-- IM SDK required start -->
    <!-- 允许程序振动，用于本地通知设置振动 -->
    <uses-permission android:name="android.permission.VIBRATE" />
    <!-- 访问网络权限 -->
    <uses-permission android:name="android.permission.INTERNET" />
    <!-- 麦克风权限，用于语音消息时录制语音，不使用录制语音可以移除 -->
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    <!-- 相机权限，用于图片消息时拍摄图片，不使用拍照可以移除 -->
    <uses-permission android:name="android.permission.CAMERA" />
    <!-- 获取运营商信息，用于获取网络状态 -->
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
    <!-- 获取读存储权限，用于附件等的获取 -->
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <!-- 访问 GPS 定位，用于定位消息，如果不用定位相关可以移除 -->
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
    <!-- 允许程序在手机屏幕关闭后后台进程仍然运行 -->
    <uses-permission android:name="android.permission.WAKE_LOCK" />
    <!-- 申请闹钟定时权限，SDK 心跳中使用，3.9.8及以后版本可以不添加 -->
    <uses-permission android:name="android.permission.SCHEDULE_EXACT_ALARM" />
    <!-- IM SDK required end -->
    <application>

        <!-- 声明 SDK 所需的 service 的核心功能-->
        <service
            android:name="com.hyphenate.chat.EMChatService"
            android:exported="true" />
        <service
            android:name="com.hyphenate.chat.EMJobService"
            android:exported="true"
            android:permission="android.permission.BIND_JOB_SERVICE" />
    </application>
</manifest>

```

- 编写index.uts 调用安卓SDK，封装SDK初始化，登录，注册，收发消息功能

```

import {
    UTSAndroid
} from "io.dcloud.uts";
import Application from 'android.app.Application'
import EMOptions from "com.hyphenate.chat.EMOptions";
import EMClient from "com.hyphenate.chat.EMClient";
import EMCallBack from 'com.hyphenate.EMCallBack';
import EMConnectionListener from 'com.hyphenate.EMConnectionListener';
import EMMessage from 'com.hyphenate.chat.EMMessage';
import EMMessageListener from 'com.hyphenate.EMMessageListener';
import EMTextMessageBody from 'com.hyphenate.chat.EMTextMessageBody';
import HyphenateException from 'com.hyphenate.exceptions.HyphenateException';
/**
 * 初始化SDK

```

```

    */
export function initChat(appkey : string) : void {
    const options = new EMOptions();
    // 设置环信AppKey
    options.setAppKey(appkey);
    // 关闭自动登录
    options.setAutoLogin(false);
    console.log(options.getAppKey(), 'getappkey')
    EMClient.getInstance().init(UTSAndroid.getAppContext(), options)
    EMClient.getInstance().addConnectionListener(new ConnectionListener());
    EMClient.getInstance().chatManager().addMessageListener(new MessageListener());
}
/**
 * 注册
 */
export function createUser(userId : string, password : string) : void {
    console.log(userId, password, 'createUser params')
    new CreateAccountThread(userId, password).start();
}
/**
 * 登录
 */
export function loginEM(userId : string, password : string) : void {
    console.log(userId, password, 'login params')
    EMClient.getInstance().login(userId, password, new LoginEMCallBack())
}
/**
 * 登出
 */
export function logoutEM() {
    EMClient.getInstance().logout(true, new LogoutEMCallBack());
}
/**
 * 发送文本消息
 */
export function sendTextMessage(msg : string, to : string) {
    const message = EMMessage.createTxtSendMessage(msg, to)
    EMClient.getInstance().chatManager().sendMessage(message);
}
/**
 * EM Login回调实现类
 */
class LoginEMCallBack extends EMCallBack {
    constructor() {
        super();
    }
    override onError(params0 : Int, params1 : string) : void {
        console.log('login error', params0, params1)
    }
    override onProgress(params0 : Int, params1 : string) : void {
        console.log('login progress', params0, params1)
    }
    override onSuccess() : void {
        console.log('login success')
    }
}
/**
 * EM Logout 回调实现类
 */

```

```

class LogoutEMCallBack extends EMCallBack {
  constructor() {
    super();
  }
  override onError(params0 : Int, params1 : string) : void {
    console.log('logout error', params0, params1)
  }
  override onSuccess() : void {
    console.log('logout success')
  }
}
/**
 * EM 连接状态 回调实现类
 */
class ConnectionListener extends EMConnectionListener {
  constructor() {
    super();
  }
  override onConnected() : void {
    console.log('onConnected 连接成功')
  }
  override onDisconnected(params0 : Int) : void {
    console.log('onDisconnected 连接断开', params0)
  }
}
/**
 * EM 消息 回调实现类
 */
class MessageListener extends EMMessageListener {
  constructor() {
    super();
  }
  override onMessageReceived(messages : MutableList<EMMessage>) : void {
    console.log('onMessageReceived 收到消息')
    messages.map((msg : EMMessage) => {
      const type = msg.getType();
      const msgBody = msg.getBody();
      console.log(`msgId: ${msg.getMsgId()}, msgFrom:${msg.getFrom()}, type:${type}`)
      if (msgBody instanceof EMTextMessageBody) {
        console.log(`Text Message`, msgBody.getMessage())
      }
    })
  }
}
/**
 * 子进程调用注册
 */
class CreateAccountThread extends Thread {
  userId : string
  password : string
  constructor(userId : string, password : string) {
    super();
    this.userId = userId;
    this.password = password;
  }
  override run() {
    // 同步方法。
    try {
      EMClient.getInstance().createAccount(this.userId, this.password);
    }
  }
}

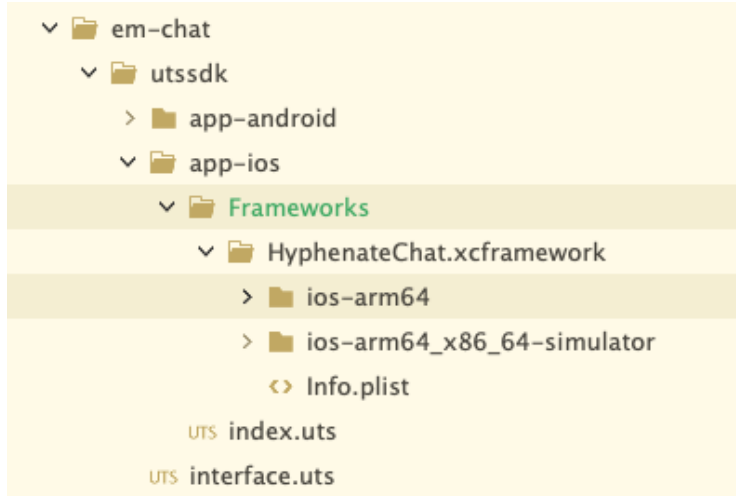
```

```
    console.log('注册成功');  
  } catch (e) {  
    const err = e as HyphenateException  
    console.log(err.getErrorCode(), err.getDescription(), '注册失败')  
  }
```

```
}  
}
```

集成 原生IOS SDK

- utssdk app-ios模块添加IOS SDK (Frameworks)



- 编写index.uts 调用 IOS SDK, 封装SDK初始化, 登录, 注册, 收发消息功能

```
import {  
    EMClient,  
    EMOptions,  
    EMError,  
    EMChatMessage,  
    EMTextMessageBody,  
    EMChatType,  
    EMChatManagerDelegate,  
    EMClientDelegate,  
    EMMessageBodyType,  
} from "HyphenateChat";  
  
/**  
 * Message事件  
 */  
class ChatMessageDelegate implements EMChatManagerDelegate {  
    messagesDidReceive(aMessages : EMChatMessage[]) : void {  
        console.log('onMessageReceived 收到消息')  
        aMessages.forEach((item : EMChatMessage) => {  
            console.log(`msgId: ${item.messageId},msgFrom:${item.from}`)  
            if (item.body.type == EMMessageBodyType.text) {  
                console.log((item.body as EMTextMessageBody).text, '文本消息');  
            }  
        })  
    }  
}  
  
let messageDelegate : ChatMessageDelegate = new ChatMessageDelegate();  
  
class EMChat {  
    emClientManager! : EMClient  
    emOptions! : EMOptions  
    constructor() {  
        this.emClientManager = EMClient.shared();  
    }  
}
```

```

    }
    initChat(appkey : string) {
        this.emOptions = new EMOptions(appkey = appkey);
        console.log(appkey, 'getappkey')
        // 初始化SDK
        this.emClientManager.initializeSDK(with = this.emOptions)
        // 添加消息delegate事件
        this.emClientManager.chatManager?.add(messageDelegate, delegateQueue = null)
    }
    login(userId : string, password : string) {
        console.log(userId, password, 'login params')
        this.emClientManager.login(
            withUsername = userId,
            password = password,
            completion = (aUserName : string, aError : EMError | null) : void => {
                if (aError === null) {
                    console.log('login success');
                    return;
                }
                console.log('login error', aError)
            }
        )
    }
    register(userId : string, password : string) {
        console.log(userId, password, 'createUser params')
        this.emClientManager.register(
            withUsername = userId,
            password = password,
            completion = (aUserName : string, aError : EMError | null) : void => {
                if (aError === null) {
                    console.log('注册成功');
                    return;
                }
                console.log('注册失败', aError)
            }
        );
    }
}
const EMChatTool : EMChat = new EMChat()

/**
 * 初始化
 */
export function initChat(appkey : string) : void {
    EMChatTool.initChat(appkey);
}

/**
 * 注册
 */
export function createUser(userId : string, password : string) : void {
    EMChatTool.register(userId, password)
}

/**
 * 登录
 */
export function loginEM(userId : string, password : string) : void {
    EMChatTool.login(userId, password)
}

```

```
/**
 * 发送文本消息
 */
export function sendTextMessage(msg : string, to : string) {
  // 消息扩展
  const ext = new Map<string, string>();
  ext.set('key', 'value')
  // 消息Body
  const body = new EMTextMessageBody(text = msg);
  const message = new EMChatMessage(
    conversationID = to,
    from = 'sttest',
    to = to,
    body = body,
    ext = ext
  );
  message.chatType = EMChatType.chat;
  EMChatTool.emClientManager.chatManager?.send(
    message,
    progress = null,
    completion = (aMessage, aError : EMError | null) : void => {
      if (aError === null) {
        console.log('send message success', aMessage);
        return;
      }
      console.log('send message failed', aError)
    }
  )
}
/**
 * 登出
 */
export function logoutEM() {
  const res : EMError | null = EMChatTool.emClientManager.logout(false);
  if (res == null) {
    console.log('logout success')
  }
}
```

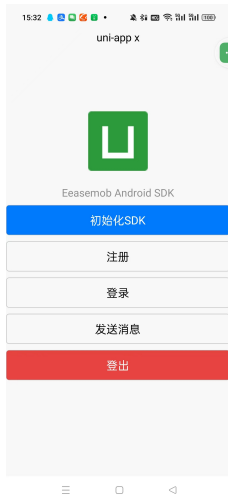


```
}  
}
```

index.uvue集成em-chat uts插件

```
<script lang="uts">  
import { initChat, loginEM, logoutEM, sendTextMessage, createUser } from "../../uni_modules/em-chat"  
export default {  
  data() {  
    return {  
      title: 'Easemob UTS SDK',  
    }  
  },  
  onLoad() {  
  },  
  methods: {  
    init() {  
      initChat(' easemob-demo#support')  
    },  
    createAccount() {  
      createUser('uniappx', '123')  
    },  
    login() {  
      loginEM('sttest', '123');  
    },  
    logout() {  
      logoutEM();  
    },  
    sendMessage() {  
      sendTextMessage(' 你好啊', 'yjj');  
    }  
  }  
}  
</script>
```

效果:



安卓

```
15:27:36.090 easemob-demo#support, getappkey at uni_modules/em-chat/utssdk/app-android/index.uts:23
15:27:36.197 onDisconnected 连接断开, [Number] 2 at uni_modules/em-chat/utssdk/app-android/index.uts:115
15:27:44.179 uniappx1, 123, createUser params at uni_modules/em-chat/utssdk/app-android/index.uts:33
15:27:44.424 注册成功 at uni_modules/em-chat/utssdk/app-android/index.uts:157
15:27:49.419 sttest, 123, login params at uni_modules/em-chat/utssdk/app-android/index.uts:41
15:27:50.150 onConnected 连接成功 at uni_modules/em-chat/utssdk/app-android/index.uts:111
15:27:50.198 login success at uni_modules/em-chat/utssdk/app-android/index.uts:77
15:27:59.490 onMessageReceived 收到消息 at uni_modules/em-chat/utssdk/app-android/index.uts:129
15:27:59.503 msgId: 1221356754609112664,msgFrom:yjj,type:TXT at uni_modules/em-chat/utssdk/app-android/index.uts:133
15:27:59.503 Text Message, 哈哈 at uni_modules/em-chat/utssdk/app-android/index.uts:135
15:28:05.065 logout success at uni_modules/em-chat/utssdk/app-android/index.uts:96
```

IOS

```
-----
16:08:29.919 easemob-demo#support, getappkey, at uni_modules/em-chat/utssdk/app-ios/index.uts:43
16:08:31.944 sttest, 123, login params, at uni_modules/em-chat/utssdk/app-ios/index.uts:51
16:08:32.962 login success, at uni_modules/em-chat/utssdk/app-ios/index.uts:57
16:08:33.989 send message success, {"receiverList":[],"body":{"isReadAcked":false,"priority":1,"localTime":17225861
"isRead":true,"isChatThreadMessage":false,"isNeedGroupAck":false,"direction":0,"deliverOnlineOnly":false,"messageId":
,"ext":{"key":"value"},"from":"sttest"}, at uni_modules/em-chat/utssdk/app-ios/index.uts:136
16:08:41.044 onMessageReceived 收到消息, at uni_modules/em-chat/utssdk/app-ios/index.uts:21
16:08:41.045 msgId: 1310056597241726508,msgFrom:yjj, at uni_modules/em-chat/utssdk/app-ios/index.uts:23
16:08:41.045 hahah&nbsp;, 文本消息, at uni_modules/em-chat/utssdk/app-ios/index.uts:25
```

结论: Uniapp x 推荐使用原生库, 支持集成原生SDK, 编写UTS插件, 不支持环信 js sdk(webview 无法传递函数等), 遇到 uts 类型问题, 打包自定义基座不稳定等

参考链接

uniapp x 介绍: <https://uniapp.dcloud.net.cn/uni-app-x/>

uts 插件介绍: <https://uniapp.dcloud.net.cn/plugin/uts-plugin.html>

uts 语言介绍: <https://uniapp.dcloud.net.cn/uts/>